

運用 Google 試算表和簡報，將大數據轉化為洞察資料

來源：<https://codelabs.developers.google.com/codelabs/bigquery-sheets-slides?hl=zh-tw>

步驟數：9

這個中階 Google Apps Script 程式碼研究室會使用 2 個 Google 開發人員平台，分別是 Google Workspace 和 Google Cloud 控制台。具體來說，這項服務會使用 Cloud 控制台的 BigQuery API (做為 Apps Script 進階服務)，以及一組內建的 Google Workspace 服務：Google 試算表和 Google 簡報。這個範例應用程式的目的是向使用者展示，他們只要透過一段簡短的程式碼，就能自動處理從大數據分析到投影片簡報的最終階段。

目錄

1. [從大數據分析到投影片簡報製作](#)
2. [問卷調查](#)
3. [總覽](#)
4. [工作 1：執行 BigQuery 並將結果記錄至試算表](#)
5. [工作 2：在 Google 試算表中建立圖表](#)
6. [工作 3：將結果資料放入簡報](#)
7. [結論](#)
8. [其他資源](#)
9. [下一步：程式碼挑戰](#)

1. 從大數據分析到投影片簡報製作

資料科學家可使用許多工具分析大數據，但最終仍須向管理團隊說明分析結果。紙上或資料庫中的大量數字，很難向主要利害關係人呈現。本中階 [Google Apps Script Codelab](#) 將使用 [Google Workspace](#) 和 [Google Cloud Console](#) 這兩個 Google 開發人員平台，協助您自動完成最後階段的作業。

Google Cloud 的開發人員工具可讓您執行深入的資料分析。接著，您可以將結果插入試算表，並使用這些資料製作投影片簡報。這項功能可讓您以更合適的方式將資料提供給管理層。本程式碼研究室會介紹 Cloud Console 的 [BigQuery API](#) (Apps Script 的[進階服務](#))，以及 [Google 試算表](#)和 [Google 簡報內建的 Apps Script 服務](#)。

動機

本程式碼研究室中的範例應用程式是參考下列程式碼範例所設計：

- [Google Apps Script BigQuery 服務範例應用程式](#)，已在 [GitHub](#) 上開放原始碼。
- [這部開發人員影片](#)和[這篇網誌文章](#)中介紹的範例應用程式。
- [Google Slides API Codelab](#) 中提供的範例應用程式。

雖然 Slides API 程式碼研究室範例應用程式也提供 BigQuery 和 Google 簡報功能，但與本程式碼研究室的範例應用程式有幾項差異：

- 他們的 Node.js 應用程式與我們的 Apps Script 應用程式。
- 他們使用 REST API，而我們使用 Apps Script 服務。
- 他們使用 Google 雲端硬碟，但未使用 Google 試算表；而這個應用程式使用 Google 試算表，但未使用 Google 雲端硬碟。

在本程式碼研究室中，我們希望將多項技術整合到單一應用程式中，同時以類似實際應用情境的方式，展示 Google Cloud 的各項功能和 API。目標是激發您的想像力，並考慮使用 Cloud Console 和 Google Workspace 解決貴機構和客戶的棘手問題。

課程內容

- 如何將 Google Apps Script 與多項 Google 服務搭配使用
- 如何使用 Google BigQuery 分析大數據
- 如何建立 Google 試算表並插入資料
- 如何在 Google 試算表中建立圖表
- 如何將 Google 試算表的資料和圖表轉移到 Google 簡報

軟硬體需求

- 可連上網際網路的網路瀏覽器
 - Google 帳戶 (Google Workspace 帳戶可能需要管理員核准)
 - 熟悉 [Google 試算表](#)的基本概念
 - 可讀取試算表 [A1 標記](#)
 - 基本 JavaScript 技能
 - 瞭解 Apps Script 開發會有幫助，但並非必要條件
-

步驟 2 · 約 0 分鐘

2. 問卷調查

3. 總覽

現在您已瞭解本程式碼研究室的主題，接下來要進行以下操作：

1. 使用現有的 Apps Script-BigQuery 範例，並讓其正常運作。
2. 從範例中瞭解如何將查詢傳送至 BigQuery 並取得結果。
3. 建立 Google 試算表，並將 BigQuery 的結果插入其中。
4. 修改程式碼，稍微變更傳回並插入試算表的資料。
5. 使用 Apps Script 中的試算表服務，為 BigQuery 資料建立圖表。
6. 使用 Google 簡報服務建立 Google 簡報。
7. 在預設標題投影片中新增標題和副標題。
8. 建立含有資料表的投影片，並將試算表的資料格匯出至投影片。
9. 建立另一張投影片，然後將試算表圖表插入其中。

首先，請先瞭解 Apps Script、BigQuery、試算表和簡報的背景資訊。

Google Apps Script 和 BigQuery

[Google Apps Script](#) 是 Google Workspace 開發平台，比起 Google REST API，能提供更高層次的操作方式。無論開發人員的技術程度如何，都能使用這個無伺服器開發與應用程式代管環境。簡單來說，Apps Script 是無伺服器 JavaScript 執行階段，可自動執行、擴充及整合 Google Workspace 功能。

Apps Script 屬於伺服器端的 JavaScript，類似 Node.js，但著重於與 Google Workspace 和其他 Google 服務緊密整合，而非快速的非同步事件導向應用程式代管。此外，開發環境可能與您以往慣用的環境不同。Apps Script 能讓您：

- 透過瀏覽器上的程式碼編輯器開發指令碼，若使用 [clasp](#) (Apps Script 的指令列部署工具)，也可選擇在本機開發。
- 以量身打造的 JavaScript 專用版本編寫程式碼，透過 Apps Script 的 [URL Fetch](#) 或 [JDBC](#) 服務存取 Google Workspace 和其他 Google 或外部服務。
- 不必編寫授權程式碼，由 Apps Script 代為處理即可。
- 不必託管應用程式，應用程式會存放在雲端上的 Google 伺服器上並在其中運行。

NOTE：本程式碼研究室不會教導您使用 Apps Script。網路上有許多資源可供參考，官方說明文件也提供[快速入門](#)、[導覽課程總覽](#)、[教學課程](#)和[影片](#)。此外，開始本程式碼研究室前，請先完成以下 2 個入門程式碼研究室：[Google 試算表適用的 Apps Script 基礎知識](#)和[動手操作 Google Apps Script](#)。

Apps Script 與其他 Google 技術的互動方式有 2 種：

- 以內建服務形式提供
- 用進階服務

[內建服務](#)提供高階方法，可與使用者資料、其他 Google 系統和外部系統互動。[進階服務](#)基本上是 Google Workspace API 或 Google REST API 的精簡 wrapper，完整支援 REST API，且功能通常比內建服務多，但需要撰寫較複雜的程式碼 (不過還是比直接使用 REST API 簡單)。進階服務在使用前，也必須在[指令碼專案中啟用](#)。

內建服務比進階服務更容易使用，且能處理更多工作，因此建議開發人員盡可能使用這類服務。不過，部分 Google API 沒有內建服務，在這種情況下，進階服務可能是唯一選擇。舉例來說，[Google BigQuery](#) 沒有內建服務，但有 [BigQuery 服務](#)。BigQuery 服務是 Cloud 控制台服務，可讓您使用 Google BigQuery API 對大型資料集 (例如數 TB 的資料) 執行查詢，幾秒內就能提供結果。

透過 Apps Script 存取試算表和簡報

與 BigQuery 不同，試算表和簡報都有內建服務。此外，您也可以使用進階服務，存取僅透過 API 提供的功能。開始編寫程式碼前，請先查看內建的 [Google 試算表](#) 和 [Google 簡報](#) 服務說明文件。請注意，[試算表](#) 和 [簡報](#) 的進階服務也有相關文件。

4. 工作 1：執行 BigQuery 並將結果記錄至試算表

簡介

我們將在本程式碼研究室中，透過第一個工作完成大部分內容。事實上，完成這個部分後，您就差不多完成了整個程式碼研究室的一半。這項主題分成幾個小節，您將瞭解：

- 建立 Google Apps Script 和 Cloud 控制台專案。
- 啟用 BigQuery 進階服務的存取權。
- 開啟指令碼編輯器，然後輸入應用程式原始碼。
- 完成應用程式授權程序 (OAuth2)。
- 執行會將要求傳送至 BigQuery 的應用程式。
- 查看使用 BigQuery 結果建立的全新 Google 試算表。

設定

1. 如要建立 Apps Script 專案，請前往 script.google.com，然後按一下「新專案」。
2. 如要重新命名 Apps Script 專案，請按一下「未命名的專案」，輸入專案標題，然後按一下「重新命名」。

接著，您需要建立 Cloud Console 專案，才能在 BigQuery 中查詢資料。

注意：如果您是在 Google 主辦活動的資訊站執行這項程式碼研究室，系統會為您建立 Cloud 控制台專案。[開啟專案](#)，然後跳至下方的步驟 5。

3. 如要建立 Cloud 控制台專案，請使用這個快速連結[建立專案](#)，為專案命名，然後按一下「建立」。

注意：BigQuery 是付費服務，儲存和查詢資料都會產生費用。本程式碼研究室查詢的資料量不大，應屬於免費用量層級。詳情請參閱 [BigQuery 定價指南](#)。

4. 專案建立完成後，頁面會顯示通知。在頁面頂端的專案清單中，確認已選取您的新專案。
5. 按一下「選單」圖示



，然後依序前往「API 和服務」>「OAuth 同意畫面」([直接連結](#))。

6. 按一下「內部」>「建立」，為貴機構的 Google Workspace 使用者建立應用程式。
7. 在「App name」(應用程式名稱) 欄位中，輸入「Big Data Codelab」。
8. 在「使用者支援」和「開發人員聯絡資訊」欄位中輸入聯絡電子郵件地址。
9. 依序點按「儲存並繼續」和「儲存並繼續」。
10. 按一下導覽列中的「更多」圖示



，然後選取「專案設定」([直接連結](#))。

11. 複製「專案編號」下方列出的值。(程式碼研究室稍後會使用另一個「專案 ID」欄位)。

注意：您也可以在 Cloud 控制台的[資訊主頁](#)中查看專案詳細資料。

接著，請將 Apps Script 專案連結至 Cloud Console 專案。

12. 切換至 App Script 編輯器，然後按一下「專案設定」圖示



。

13. 點選「Google Cloud Platform (GCP) 專案」下方的「變更專案」。

14. 輸入專案編號，然後點選「設定專案」。

15. 接著，按一下「編輯器」



，開始新增 BigQuery 進階服務。

16. 按一下「服務」旁的「新增服務」圖示



。

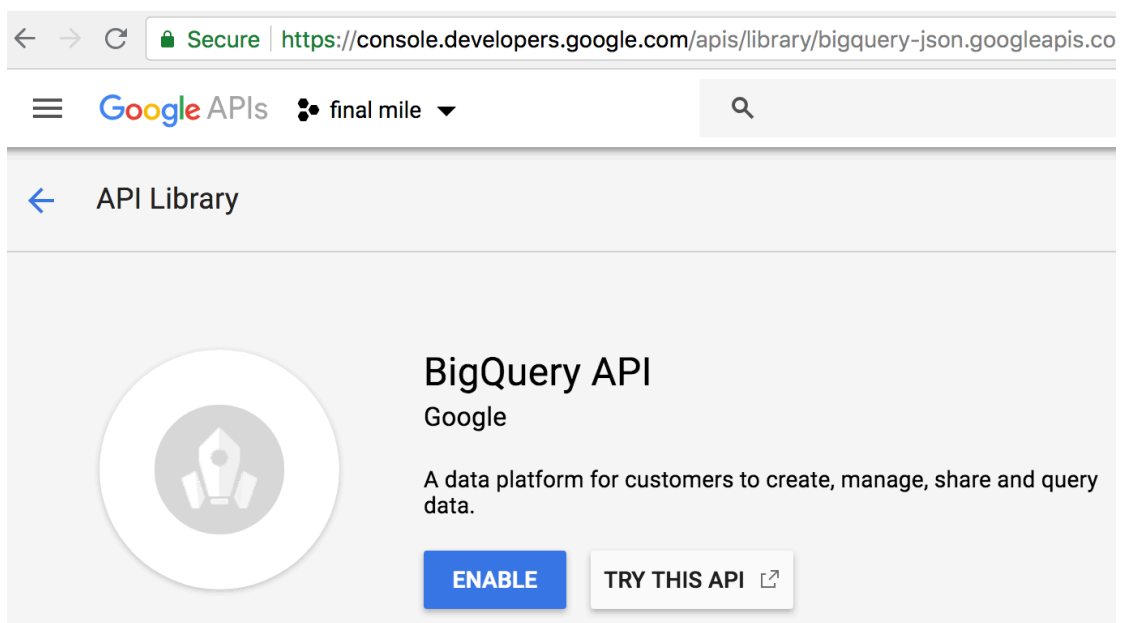
17. 在「新增服務」對話方塊中，選取「BigQuery API」，然後點選「新增」。

最後一個步驟是在 Cloud Console 中開啟 BigQuery API。

18. 如要執行這項操作，請切換至 Cloud 控制台，然後依序點選「API 和服務」>「資訊主頁」。(請確認您仍位於步驟 3 建立的專案中)。

19. 點選「啟用 API 和服務」。

20. 搜尋「big query」，選取 **BigQuery API** (不是 BigQuery Data Transfer API)，然後按一下「啟用」開啟。



NOTE：啟用 API 後，您可能在這個頁面上看到「如要使用這個 API，請建立憑證...」的附註。目前不必擔心這個問題，Apps Script 會為您處理這個步驟。

現在您可以輸入應用程式程式碼、完成授權程序，讓這個應用程式的第一個疊代版本開始運作。

上傳並執行應用程式

1. 在指令碼編輯器中，將預設的 `myFunction()` 程式碼區塊替換為下列程式碼：

```

// File name for data results
var QUERY_NAME = "Most common words in all of Shakespeare's works";
// Replace this value with your Google Cloud Console Project ID
var PROJECT_ID = '';
if (!PROJECT_ID) throw Error('Project ID is required in setup');

/**
 * Runs a BigQuery query; puts results into a Sheet. You must turn on
 * the BigQuery advanced service before you can run this code.
 * @see https://developers.google.com/apps-script/advanced/bigquery#run\_query
 * @see https://github.com/googleworkspace/apps-script-samples/blob/master/advanced/bigquery.gs
 *
 * @returns {Spreadsheet} Returns a spreadsheet with BigQuery results
 * @see https://developers.google.com/apps-script/reference/spreadsheet/spreadsheet
 */
function runQuery() {
  // Replace sample with your own BigQuery query.
  var request = {
    query:
      'SELECT ' +
        'LOWER(word) AS word, ' +
        'SUM(word_count) AS count ' +
      'FROM [bigquery-public-data:samples.shakespeare] ' +
      'GROUP BY word ' +
      'ORDER BY count ' +
      'DESC LIMIT 10'
  };

  var queryResults = BigQuery.Jobs.query(request, PROJECT_ID);
  var jobId = queryResults.jobReference.jobId;

  // Wait for BigQuery job completion (with exponential backoff).
  var sleepTimeMs = 500;
  while (!queryResults.jobComplete) {
    Utilities.sleep(sleepTimeMs);
    sleepTimeMs *= 2;
    queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId);
  }

  // Get all results from BigQuery.
  var rows = queryResults.rows;
  while (queryResults.pageToken) {
    queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId, {
      pageToken: queryResults.pageToken
    });
    rows = rows.concat(queryResults.rows);
  }

  // Return null if no data returned.
  if (!rows) {
    return Logger.log('No rows returned.');
```

```

// Create the results spreadsheet.
var spreadsheet = SpreadsheetApp.create(QUERY_NAME);
var sheet = spreadsheet.getActiveSheet();

// Add headers to Sheet.
var headers = queryResults.schema.fields.map(function(field) {
  return field.name.toUpperCase();
});
sheet.appendRow(headers);

// Append the results.
var data = new Array(rows.length);
for (var i = 0; i < rows.length; i++) {
  var cols = rows[i].f;
  data[i] = new Array(cols.length);
  for (var j = 0; j < cols.length; j++) {
    data[i][j] = cols[j].v;
  }
}

// Start storing data in row 2, col 1
var START_ROW = 2;      // skip header row
var START_COL = 1;
sheet.getRange(START_ROW, START_COL, rows.length, headers.length).setValues(data);

Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl());
}

```

2. 按一下「Save」(儲存) 圖示



。

3. 在 Code.gs 旁邊，依序點選「更多」圖示



> 「重新命名」。將標題從「Code.gs」變更為「bq-sheets-slides.js」。

4. 接著，我們來看看查詢 BigQuery 並將結果寫入 Google 試算表的程式碼。您可以在 runQuery() 頂端附近查看：

```

SELECT
  LOWER(word) AS word,
  SUM(word_count) AS count
FROM [bigquery-public-data:samples.shakespeare]
GROUP BY word
ORDER BY count
DESC LIMIT 10

```

這項查詢會檢索 [BigQuery 公開資料集](#) 中的所有莎士比亞作品，找出出現頻率最高的前 10 個字詞，並依熱門程度從高到低排序。想像一下手動彙整這些資料需要多少工作量，就能瞭解 BigQuery 有多實用。

函式也會宣告 `PROJECT_ID` 變數，該變數需要有效的 Cloud 控制台專案 ID。變數下方的 `if` 陳述式可防止應用程式在沒有專案 ID 的情況下繼續執行。

5. 切換至 Cloud 控制台專案，按一下導覽列中的「更多」圖示



，然後選取「專案設定」。

6. 複製「專案 ID」下方列出的值。

7. 切換回 Apps Script 編輯器，在 `PROJECT_ID` 中找出 `bq-sheets-slides.js` 變數，然後新增值。

8. 依序點選「儲存」



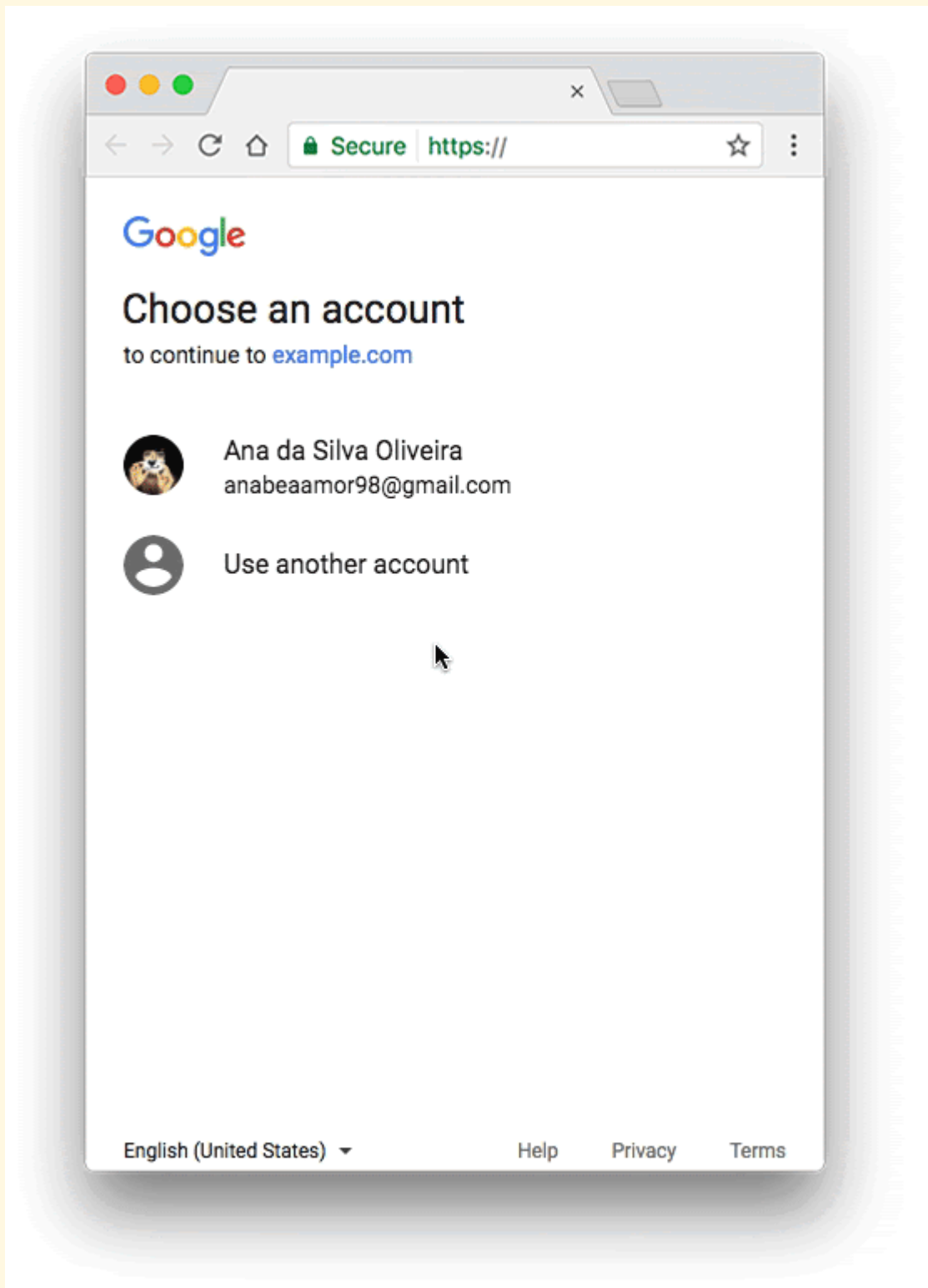
>「執行」。

9. 按一下「查看權限」繼續操作。

注意：如果您使用 gmail.com 帳戶，第一次使用指令碼時可能會看到「這個應用程式未經驗證」對話方塊。Google 會使用這項資訊，警告可能使用不明或不受信任作者程式碼的使用者。如果您是指令碼作者，看到這個對話方塊時可以繼續操作。按照畫面上的提示繼續授權指令碼：

1. 在未經驗證的應用程式對話方塊中，按一下「進階」。
2. 按一下「前往『your-project-name』(不安全)」。
3. 按一下「允許」。

在本程式碼研究室中，您可能會收到多個權限提示。如要進一步瞭解這項程序，請參閱「[Google 服務授權](#)」。

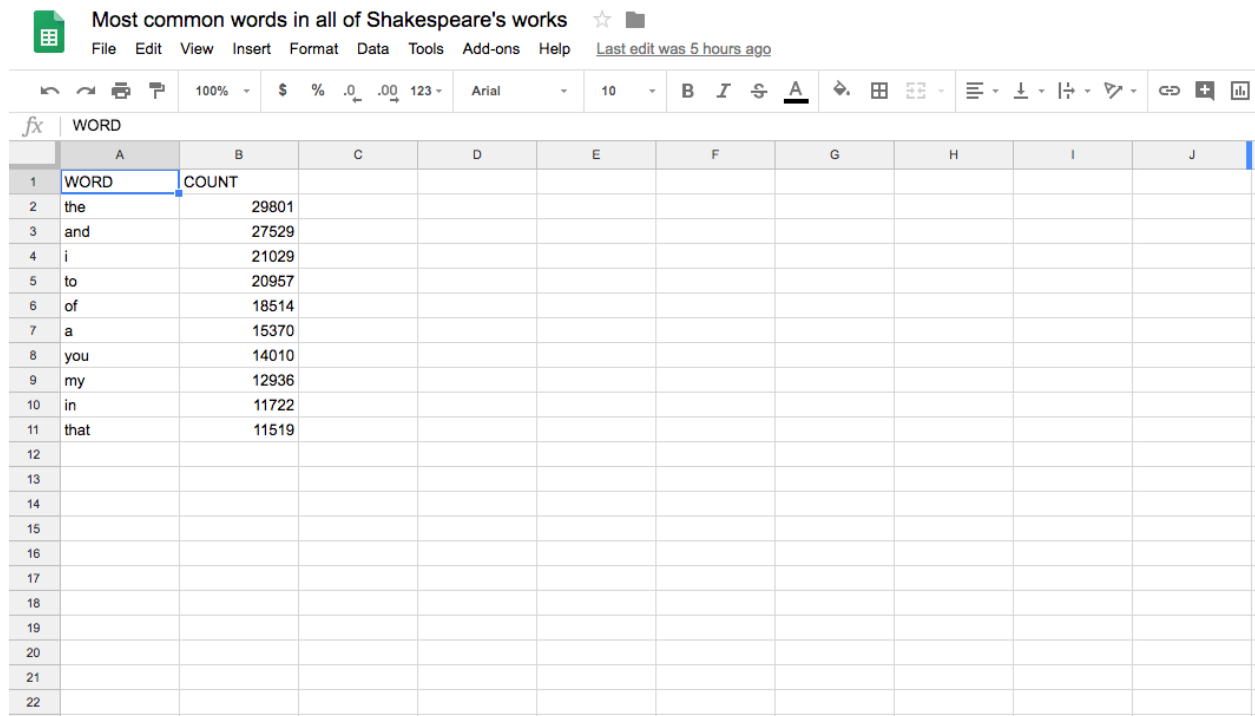


10. 指令碼開始執行後，系統會開啟內建的執行記錄，並即時記錄指令碼動作。

如要顯示或隱藏執行作業記錄，請按一下編輯器頂端的「執行作業記錄」。

11. 執行記錄顯示「Execution completed」後，請前往 Google 雲端硬碟 (drive.google.com)，找出名為「Most common words in all of Shakespeare's works」的 Google 試算表 (或您指派給 `QUERY_NAME` 變數的名稱，如果已更新)：

12. 開啟試算表，即可查看 10 個最常見的字詞，以及依出現次數由高到低排列的總次數：



Most common words in all of Shakespeare's works										
File Edit View Insert Format Data Tools Add-ons Help Last edit was 5 hours ago										
fx WORD										
	A	B	C	D	E	F	G	H	I	J
1	WORD	COUNT								
2	the	29801								
3	and	27529								
4	i	21029								
5	to	20957								
6	of	18514								
7	a	15370								
8	you	14010								
9	my	12936								
10	in	11722								
11	that	11519								
12										
13										
14										
15										
16										
17										
18										
19										
20										
21										
22										

工作 1 摘要

回顧一下，您執行了一些程式碼，查詢莎士比亞的所有作品，並查看每部戲劇中的每個字。並依出現次數遞減排序。您也使用了 Google 試算表的 Apps Script 內建服務來顯示這項資料。

您也可以在 github.com/googlecode/bigquery-sheets-slides 的程式碼研究室 GitHub 存放區中，找到 `bq-sheets-slides.js` 的 [step1](#) 資料夾，這段程式碼的靈感來自 [BigQuery 進階服務頁面中的原始範例](#)，該範例執行了略有不同的查詢，擷取莎士比亞使用過且長度超過 10 個字元的熱門字詞。您也可以在 GitHub 存放區中查看範例。

如要瞭解如何使用莎士比亞作品或其他公開資料表建構其他查詢，請參閱「[如何查詢 BigQuery 範例資料表](#)」和 [這個 GitHub 存放區](#)。

您也可以先在 Cloud Console 的 BigQuery 頁面中執行查詢，再於 Apps Script 中執行。如要尋找，請按一下「選單」圖示



，然後前往「BigQuery UI」>「SQL 工作區」([直接連結](#))。舉例來說，我們的查詢在 BigQuery 圖形介面中會顯示如下：

Google Cloud Platform

FEATURES & INFO

SHORTCUT

HIDE PREVIEW FEATURES

Explorer

Type to search

Viewing pinned projects.

*UNSAVE...

RUN

SAVE

SCHEDULE

MORE

1

2

3

4

5

SELECT LOWER(word) AS word, SUM(word_count) AS count

FROM bigquery-public-data.samples.shakespeare

GROUP BY word

ORDER BY count

DESC LIMIT 10

Query results

SAVE RESULTS

EXPLORE DATA

Query complete (0.0 sec elapsed, cached)

Job information

Results

JSON

Row	word	count
1	the	29801
2	and	27529
3	i	21029
4	to	20957
5	of	18514
6	a	15370
7	you	14010
8	my	12936
9	in	11722
10	that	11519

5. 工作 2：在 Google 試算表中建立圖表

`runQuery()` 的用途是使用 BigQuery，並將資料結果傳送至 Google 試算表。接著，我們需要使用資料製作圖表。建立名為 `createColumnChart()` 的新函式，呼叫 Google 試算表的 `newChart()` 方法。

1. 在 Apps Script 編輯器中，將 `createColumnChart()` 函式新增至 `runQuery()` 後方的 `bq-sheets-slides.js`。這段程式碼會取得試算表，並要求系統建立涵蓋所有資料的柱狀圖。由於第一列是欄標題，因此資料範圍從儲存格 A2 開始。

```
/**
 * Uses spreadsheet data to create a column chart.
 * @param {Spreadsheet} Spreadsheet containing results data
 * @returns {EmbeddedChart} Visualizes the results
 * @see https://developers.google.com/apps-script/reference/spreadsheet/embedded-chart
 */
function createColumnChart(spreadsheet) {
  // Retrieve the populated (first) Sheet.
  var sheet = spreadsheet.getSheets()[0];
  // Data range in Sheet is from cell A2 to B11.
  var START_CELL = 'A2'; // skip header row
  var END_CELL = 'B11';
  // Place chart on Sheet starting on cell E5.
  var START_ROW = 5;      // row 5
  var START_COL = 5;      // col E
  var OFFSET = 0;

  // Create & place chart on the Sheet using above values.
  var chart = sheet.newChart()
    .setChartType(Charts.ChartType.COLUMN)
    .addRange(sheet.getRange(START_CELL + ':' + END_CELL))
    .setPosition(START_ROW, START_COL, OFFSET, OFFSET)
    .build();
  sheet.insertChart(chart);
}
```

2. `createColumnChart()` 函式需要試算表物件參數，因此我們需要更新 `runQuery()`，傳回可傳遞至 `createColumnChart()` 的 `spreadsheet` 物件。在 `runQuery()` 結尾，記錄試算表成功建立後，傳回 `spreadsheet` 物件：

```
Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl());

// NEW: Return the spreadsheet object for later use.
return spreadsheet;
}
```

3. 建立 `createBigQueryPresentation()` 函式，同時呼叫 `runQuery()` 和 `createColumnChart()`。在邏輯上將 BigQuery 和圖表建立功能分開是最佳做法：


```
/**
 * Runs the query, adds data and a chart to a spreadsheet.
 */
function createBigQueryPresentation() {
  var spreadsheet = runQuery();
  createColumnChart(spreadsheet);
}
```

4. 您已完成上述 2 個重要步驟，也就是傳回試算表物件，以及建立輸入函式。為了提高 `runQuery()` 的可用性，我們需要將記錄檔行從 `runQuery()` 移至 `createBigQueryPresentation()`。現在的方法應如下所示：

```
/**
 * Runs a BigQuery query, adds data and a chart to a spreadsheet.
 */
function createBigQueryPresentation() {
  var spreadsheet = runQuery();
  Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl()); // MOVED HERE
  createColumnChart(spreadsheet);
}
```

完成上述變更後 (`PROJECT_ID` 除外)，`bq-sheets-slides.js` 應如下所示。您也可以在此 [GitHub 存放區的 step2](#) 中找到這段程式碼。

```

// File name for data results
var QUERY_NAME = "Most common words in all of Shakespeare's works";
// Replace this value with your Google Cloud Console Project ID
var PROJECT_ID = '';
if (!PROJECT_ID) throw Error('Project ID is required in setup');

/**
 * Runs a BigQuery query; puts results into a sheet. You must turn on
 * the BigQuery advanced service before you can run this code.
 * @see https://developers.google.com/apps-script/advanced/bigquery#run_query
 * @see https://github.com/googleworkspace/apps-script-
samples/blob/master/advanced/bigquery.gs
 *
 * @returns {Spreadsheet} Returns a spreadsheet with BigQuery results
 * @see https://developers.google.com/apps-script/reference/spreadsheet/spreadsheet
 */
function runQuery() {
  // Replace sample with your own BigQuery query.
  var request = {
    query:
      'SELECT ' +
        'LOWER(word) AS word, ' +
        'SUM(word_count) AS count ' +
      'FROM [bigquery-public-data:samples.shakespeare] ' +
      'GROUP BY word ' +
      'ORDER BY count ' +
      'DESC LIMIT 10'
  };

  var queryResults = BigQuery.Jobs.query(request, PROJECT_ID);
  var jobId = queryResults.jobReference.jobId;

  // Wait for BigQuery job completion (with exponential backoff).
  var sleepTimeMs = 500;
  while (!queryResults.jobComplete) {
    Utilities.sleep(sleepTimeMs);
    sleepTimeMs *= 2;
    queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId);
  }

  // Get all results from BigQuery.
  var rows = queryResults.rows;
  while (queryResults.pageToken) {
    queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId, {
      pageToken: queryResults.pageToken
    });
    rows = rows.concat(queryResults.rows);
  }

  // Return null if no data returned.
  if (!rows) {
    return Logger.log('No rows returned.');
```

```

// Create the results spreadsheet.
var spreadsheet = SpreadsheetApp.create(QUERY_NAME);
var sheet = spreadsheet.getActiveSheet();

// Add headers to sheet.
var headers = queryResults.schema.fields.map(function(field) {
    return field.name.toUpperCase();
});
sheet.appendRow(headers);

// Append the results.
var data = new Array(rows.length);
for (var i = 0; i < rows.length; i++) {
    var cols = rows[i].f;
    data[i] = new Array(cols.length);
    for (var j = 0; j < cols.length; j++) {
        data[i][j] = cols[j].v;
    }
}

// Start storing data in row 2, col 1
var START_ROW = 2;      // skip header row
var START_COL = 1;
sheet.getRange(START_ROW, START_COL, rows.length, headers.length).setValues(data);

// Return the spreadsheet object for later use.
return spreadsheet;
}

/**
 * Uses spreadsheet data to create a columnar chart.
 * @param {Spreadsheet} Spreadsheet containing results data
 * @returns {EmbeddedChart} Visualizes the results
 * @see https://developers.google.com/apps-script/reference/spreadsheet/embedded-chart
 */
function createColumnChart(spreadsheet) {
    // Retrieve the populated (first) sheet.
    var sheet = spreadsheet.getSheets()[0];
    // Data range in sheet is from cell A2 to B11.
    var START_CELL = 'A2'; // skip header row
    var END_CELL = 'B11';
    // Place chart on Sheet starting on cell E5.
    var START_ROW = 5;      // row 5
    var START_COL = 5;      // col E
    var OFFSET = 0;

    // Create & place chart on the sheet using above values.
    var chart = sheet.newChart()
        .setChartType(Charts.ChartType.COLUMN)
        .addRange(sheet.getRange(START_CELL + ':' + END_CELL))
        .setPosition(START_ROW, START_COL, OFFSET, OFFSET)

```

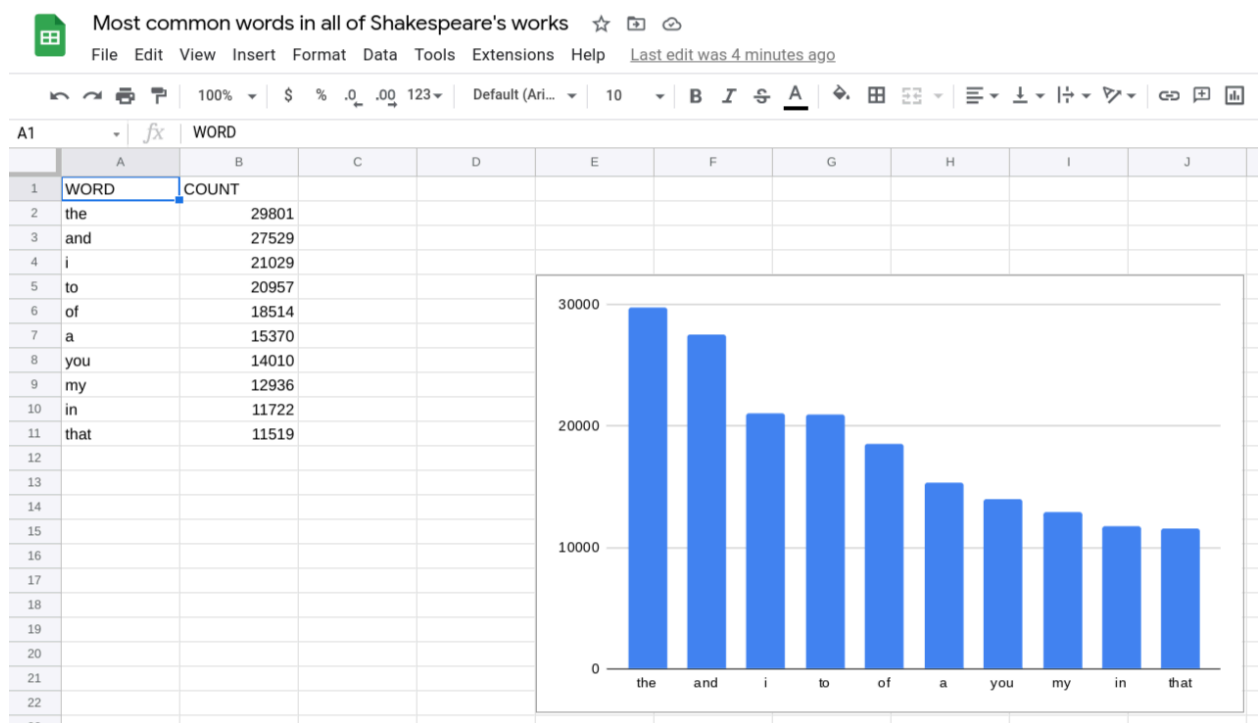
```

        .build();
        sheet.insertChart(chart);
    }

    /**
     * Runs a BigQuery query, adds data and a chart to a spreadsheet.
     */
    function createBigQueryPresentation() {
        var spreadsheet = runQuery();
        Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl());
        createColumnChart(spreadsheet);
    }

```

在指令碼編輯器中儲存指令碼專案。然後從函式清單中選取 `createBigQueryPresentation`，並按一下「執行」。完成後，Google 雲端硬碟中會建立另一份試算表，但這次試算表中的資料旁會顯示圖表：



6. 工作 3：將結果資料放入簡報

本程式碼研究室的最後一部分是建立 Google 簡報檔案、在標題投影片上新增標題和副標題，然後為資料儲存格和圖表建立投影片。

1. 在 Apps Script 編輯器中，將 `createSlidePresentation()` 函式新增至 `createColumnChart()` 後方的 `bq-sheets-slides.js`。所有投影片組作業都會在這個函式中進行。首先，請建立簡報，然後在預設的標題投影片中新增標題和副標題。

```
/**
 * Create presentation with spreadsheet data and a chart
 * @param {Spreadsheet} Spreadsheet containing results data
 * @param {EmbeddedChart} Sheets chart to embed on a slide
 * @returns {Presentation} Slide deck with the results
 * @see https://developers.google.com/apps-script/reference/slides/presentation
 */
function createSlidePresentation(spreadsheet, chart) {
  // Create the presentation.
  var deck = SlidesApp.create(QUERY_NAME);

  // Populate the title slide.
  var [title, subtitle] = deck.getSlides()[0].getPageElements();
  title.asShape().getText().setText(QUERY_NAME);
  subtitle.asShape().getText().setText('using Google Cloud Console and Google Workspace APIs:\n' +
    'Google Apps Script, BigQuery, Sheets, Slides');
```

2. 在 `createSlidePresentation()` 流程中，下一步是將 Google 試算表中的儲存格資料匯入新的簡報。將以下程式碼片段新增至函式：

```
// Data range to copy is from cell A1 to B11
var START_CELL = 'A1'; // include header row
var END_CELL = 'B11';
// Add the table slide and insert an empty table on it with
// the dimensions of the data range; fails if the sheet is empty.
var tableSlide = deck.appendSlide(SlidesApp.PredefinedLayout.BLANK);
var sheetValues = spreadsheet.getSheets()[0].getRange(
  START_CELL + ':' + END_CELL).getValues();
var table = tableSlide.insertTable(sheetValues.length, sheetValues[0].length);

// Populate the table with spreadsheet data.
for (var i = 0; i < sheetValues.length; i++) {
  for (var j = 0; j < sheetValues[0].length; j++) {
    table.getCell(i, j).getText().setText(String(sheetValues[i][j]));
  }
}
```

3. `createSlidePresentation()` 的最後一個步驟是再新增一張投影片、匯入試算表中的圖表，然後傳回 `Presentation` 物件。將以下程式碼片段新增至函式：

```
// Add a chart slide and insert the chart on it.
var chartSlide = deck.appendSlide(SlidesApp.PredefinedLayout.BLANK);
chartSlide.insertSheetsChart(chart);

// Return the presentation object for later use.
return deck;
}
```

4. 現在函式已完成，請再確認一下簽章。 `createSlidePresentation()` 需要試算表和圖表物件參數。我們已將 `runQuery()` 調整為傳回 `Spreadsheet` 物件，但需要對 `createColumnChart()` 做出類似調整，才能傳回圖表物件 (`EmbeddedChart`)。返回 `createColumnChart()`，並在函式結尾新增下列程式碼片段：

```
// NEW: Return the chart object for later use.
return chart;
}
```

5. 由於 `createColumnChart()` 現在會傳回圖表物件，因此我們需要將圖表儲存至變數。然後，我們將試算表和圖表變數一併傳遞至 `createSlidePresentation()`。此外，由於我們會記錄新建立的試算表網址，因此也請記錄新投影片簡報的網址。更新 `createBigQueryPresentation()`，使其如下所示：

```
/**
 * Runs a BigQuery query, adds data and a chart to a spreadsheet,
 * and adds the data and chart to a new slide presentation.
 */
function createBigQueryPresentation() {
  var spreadsheet = runQuery();
  Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl());
  var chart = createColumnChart(spreadsheet); // UPDATED
  var deck = createSlidePresentation(spreadsheet, chart); // NEW
  Logger.log('Results slide deck created: %s', deck.getUrl()); // NEW
}
```

6. 儲存並再次執行 `createBigQueryPresentation()`。不過，應用程式需要使用者再授予一組權限，才能查看及管理 Google 簡報檔案。查看並允許這些權限後，應用程式就會照常運作。
7. 現在除了建立的試算表外，您還會看到新的簡報檔案，內含 3 張投影片 (標題、資料表和資料圖表)，如下所示：



Most common words in all of Shakespeare's works



File Edit View Insert Format Slide Arrange Tools Add-ons Help [Last edit...](#)



PRESENT

SHARE

+



Background...

Layout

Theme...

Transition...

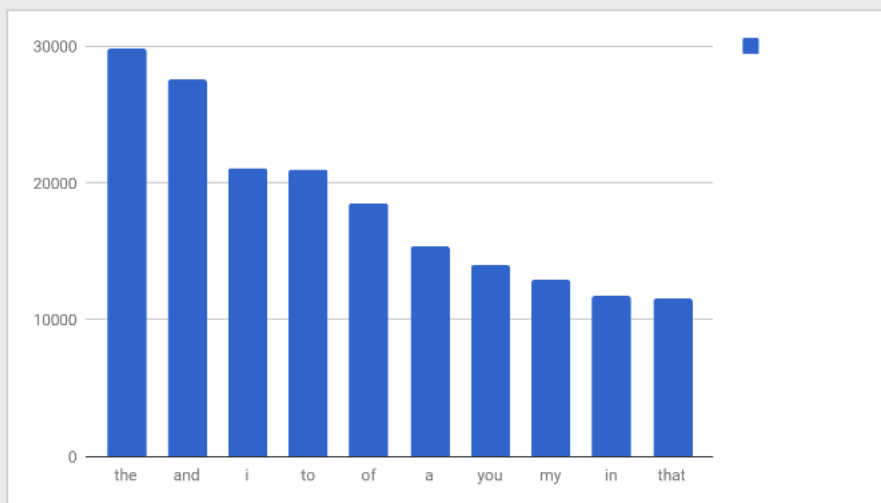
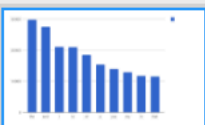
1

Most common words in all of Shakespeare's works
via GCP and G Suite APIs
Google Apps Script, BigQuery, Sheets, Slides

2

Word	Count
the	29854
and	27321
i	21111
to	21088
of	18544
a	15444
you	14000
my	12999
in	11999
that	11555

3



7. 結論

恭喜！您已建立同時使用 Google Cloud 兩端的應用程式。這項作業會執行 Google BigQuery 要求，查詢其中一個公開資料集、建立 Google 試算表來儲存結果、根據資料新增圖表，最後建立 Google 簡報，其中包含試算表的資料和圖表結果。

確切來說，是這樣沒錯。以宏觀的角度來看，就是執行大數據分析，然後將結果轉化成能向利害關係人展示的內容，而且全都是透過程式碼自動完成。希望這個範例能激發您的靈感，為自己的專案量身打造。本程式碼研究室結束時，我們會提供一些建議，說明如何進一步強化這個範例應用程式。

完成最後一項工作後，`bq-sheets-slides.js` 的內容應如下所示 (`PROJECT_ID` 除外)：

```

/**
 * Copyright 2018 Google LLC
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at apache.org/licenses/LICENSE-2.0.
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

// File name for data results
var QUERY_NAME = "Most common words in all of Shakespeare's works";
// Replace this value with your Google Cloud Console Project ID
var PROJECT_ID = '';
if (!PROJECT_ID) throw Error('Project ID is required in setup');

/**
 * Runs a BigQuery query; puts results into a spreadsheet. You must turn on
 * the BigQuery advanced service before you can run this code.
 * @see https://developers.google.com/apps-script/advanced/bigquery#run\_query
 * @see https://github.com/googleworkspace/apps-script-samples/blob/master/advanced/bigquery.gs
 *
 * @returns {Spreadsheet} Returns a spreadsheet with BigQuery results
 * @see https://developers.google.com/apps-script/reference/spreadsheet/spreadsheet
 */
function runQuery() {
  // Replace sample with your own BigQuery query.
  var request = {
    query:
      'SELECT ' +
        'LOWER(word) AS word, ' +
        'SUM(word_count) AS count ' +
      'FROM [bigquery-public-data:samples.shakespeare] ' +
      'GROUP BY word ' +
      'ORDER BY count ' +
      'DESC LIMIT 10'
  };

  var queryResults = BigQuery.Jobs.query(request, PROJECT_ID);
  var jobId = queryResults.jobReference.jobId;

  // Wait for BigQuery job completion (with exponential backoff).
  var sleepTimeMs = 500;
  while (!queryResults.jobComplete) {
    Utilities.sleep(sleepTimeMs);
    sleepTimeMs *= 2;
    queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId);
  }
}

```

```

// Get all results from BigQuery.
var rows = queryResults.rows;
while (queryResults.pageToken) {
  queryResults = BigQuery.Jobs.getQueryResults(PROJECT_ID, jobId, {
    pageToken: queryResults.pageToken
  });
  rows = rows.concat(queryResults.rows);
}

// Return null if no data returned.
if (!rows) {
  return Logger.log('No rows returned.');
```

```

// Create the results spreadsheet.
```

```

var spreadsheet = SpreadsheetApp.create(QUERY_NAME);
var sheet = spreadsheet.getActiveSheet();
```

```

// Add headers to sheet.
```

```

var headers = queryResults.schema.fields.map(function(field) {
  return field.name.toUpperCase();
});
sheet.appendRow(headers);
```

```

// Append the results.
```

```

var data = new Array(rows.length);
for (var i = 0; i < rows.length; i++) {
  var cols = rows[i].f;
  data[i] = new Array(cols.length);
  for (var j = 0; j < cols.length; j++) {
    data[i][j] = cols[j].v;
  }
}
```

```

// Start storing data in row 2, col 1
```

```

var START_ROW = 2;      // skip header row
var START_COL = 1;
sheet.getRange(START_ROW, START_COL, rows.length, headers.length).setValues(data);
```

```

// Return the spreadsheet object for later use.
```

```

return spreadsheet;
}
```

```

/**
```

```

 * Uses spreadsheet data to create a column chart.
```

```

 * @param {Spreadsheet} Spreadsheet containing results data
```

```

 * @returns {EmbeddedChart} Visualizes the results
```

```

 * @see https://developers.google.com/apps-script/reference/spreadsheet/embedded-chart
```

```

 */
```

```

function createColumnChart(spreadsheet) {
```

```

  // Retrieve the populated (first) sheet.
```

```

var sheet = spreadsheet.getSheets()[0];
// Data range in sheet is from cell A2 to B11.
var START_CELL = 'A2'; // skip header row
var END_CELL = 'B11';
// Place chart on sheet starting on cell E5.
var START_ROW = 5;      // row 5
var START_COL = 5;      // col E
var OFFSET = 0;

// Create & place chart on the sheet using above values.
var chart = sheet.newChart()
    .setChartType(Charts.ChartType.COLUMN)
    .addRange(sheet.getRange(START_CELL + ':' + END_CELL))
    .setPosition(START_ROW, START_COL, OFFSET, OFFSET)
    .build();
sheet.insertChart(chart);

// Return the chart object for later use.
return chart;
}

/**
 * Create presentation with spreadsheet data and a chart
 * @param {Spreadsheet} Spreadsheet containing results data
 * @param {EmbeddedChart} Sheets chart to embed on a slide
 * @returns {Presentation} Slide deck with the results
 * @see https://developers.google.com/apps-script/reference/slides/presentation
 */
function createSlidePresentation(spreadsheet, chart) {
    // Create the presentation.
    var deck = SlidesApp.create(QUERY_NAME);

    // Populate the title slide.
    var [title, subtitle] = deck.getSlides()[0].getPageElements();
    title.asShape().getText().setText(QUERY_NAME);
    subtitle.asShape().getText().setText('using Google Cloud Console and Google Workspace APIs:\n' +
        'Google Apps Script, BigQuery, Sheets, Slides');

    // Data range to copy is from cell A1 to B11
    var START_CELL = 'A1'; // include header row
    var END_CELL = 'B11';
    // Add the table slide and insert an empty table on it with
    // the dimensions of the data range; fails if the sheet is empty.
    var tableSlide = deck.appendSlide(SlidesApp.PredefinedLayout.BLANK);
    var sheetValues = spreadsheet.getSheets()[0].getRange(
        START_CELL + ':' + END_CELL).getValues();
    var table = tableSlide.insertTable(sheetValues.length, sheetValues[0].length);

    // Populate the table with spreadsheet data.
    for (var i = 0; i < sheetValues.length; i++) {
        for (var j = 0; j < sheetValues[0].length; j++) {

```

```

        table.getCell(i, j).getText().setText(String(sheetValues[i][j]));
    }
}

// Add a chart slide and insert the chart on it.
var chartSlide = deck.appendSlide(SlidesApp.PredefinedLayout.BLANK);
chartSlide.insertSheetsChart(chart);

// Return the presentation object for later use.
return deck;
}

/**
 * Runs a BigQuery query, adds data and a chart to a spreadsheet,
 * and adds the data and chart to a new slide presentation.
 */
function createBigQueryPresentation() {
    var spreadsheet = runQuery();
    Logger.log('Results spreadsheet created: %s', spreadsheet.getUrl());
    var chart = createColumnChart(spreadsheet);
    var deck = createSlidePresentation(spreadsheet, chart);
    Logger.log('Results slide deck created: %s', deck.getUrl());
}

```

您也可以在 [GitHub](#) 存放區的 `final` 資料夾中找到這個程式碼範例。

8. 其他資源

您也可以參考下方的額外資源，深入瞭解本程式碼研究室涵蓋的內容，並探索其他透過程式存取 Google 開發人員工具的方法。

這個應用程式的資源

- [原始碼存放區](#)
- [開發人員影片和網誌文章](#)
- [Google Cloud NEXT '18 研討會](#)

說明文件

- [Google Apps Script 說明文件網站](#)
- [Apps Script：試算表服務](#)
- [Apps Script：簡報服務](#)
- [Apps Script：BigQuery 進階服務](#)

影片

- [另一個 Google Apps Script 秘訣](#)
- [從試算表存取 Google 地圖](#)
- [Totally Unscripted](#)
- [Google Workspace 開發人員節目](#)

最新消息與更新

- [Google Cloud Platform 網誌](#)
- [Google Cloud 資料分析網誌](#)
- [Google 開發人員網誌](#)
- Twitter：[Google Developers \(@googledevs\)](#)
- [Google Workspace 開發人員網誌](#)
- Twitter：[Google Workspace 開發人員 \(@workspacedevs\)](#)

其他程式碼研究室

入門

- [Google 試算表] [Google 試算表 Apps Script 基礎知識](#)
- [REST API] [使用 Google Workspace 和 Google API 存取 Google 雲端硬碟中的檔案和資料夾](#)

中級

- [Apps Script] [CLASP Apps Script 指令列工具](#)
- [Apps Script] [Gmail 適用的 Google Workspace 外掛程式](#)
- [Apps Script] [Hangouts Chat 的自訂機器人](#)
- [REST API] [使用 Google 試算表做為應用程式的報表工具](#)

- [REST API] [使用 BigQuery API 產生 Google 簡報](#)
-

9. 下一步：程式碼挑戰

以下列出幾種方法，可強化或擴增本程式碼研究室中建構的範例。以下僅列出部分類型，不過仍提供了後續步驟的一些靈感。

- **應用程式**：不想受限於 JavaScript 或 Apps Script 的限制嗎？將這個應用程式移植到您慣用的程式設計語言，並使用 Google BigQuery、試算表和簡報的 REST API。
 - **BigQuery**：針對您感興趣的莎士比亞資料集，嘗試不同的查詢。如需其他查詢範例，請參閱原始的 [Apps Script BigQuery 範例應用程式](#)。
 - **BigQuery**：試用其他 [BigQuery 公開資料集](#)，找出對您更有意義的資料集。
 - **BigQuery**：我們稍早提到，您可以使用莎士比亞作品或其他公開資料表建立其他查詢。您可以在[這個網頁](#)和[這個 GitHub 存放區](#)中找到這些範例。
 - **試算表**：在「圖表庫」中嘗試其他圖表類型。
 - **試算表和 BigQuery**：使用您自己的大型試算表資料集。2016 年，BigQuery 團隊推出一項功能，讓開發人員能使用試算表做為資料來源。詳情請參閱「[Google BigQuery 與 Google 雲端硬碟整合](#)」。
 - **投影片**：在生成的簡報中加入其他投影片，例如與大數據分析相關的圖片或其他資產。以下是 [Slides 內建服務](#)的參考說明文件。
 - **Google Workspace**：透過 Apps Script 使用其他 Google Workspace 或 Google 內建服務。例如 Gmail、日曆、文件、雲端硬碟、地圖、Analytics、YouTube 等，以及其他進階服務。如需更多資訊，請參閱內建和進階服務的[參考總覽](#)。
-